

Safiri ETA Prediction — Technical Report

1. Problem

A freight shipment moves through five sequential stages — origin departure, port arrival, customs clearance, inland transport, and final delivery. Time lost at an early stage propagates downstream and compresses the schedule left for every later stage, so the final delivery delay is not independent noise: it is partly determined before the shipment is halfway through its journey.

The deliverable is a system that, at the moment the vessel docks at port (the prediction cutoff), produces (a) a forecast of the final delivery delay in hours, (b) the probability that the shipment ends up "delayed", and (c) an explanation of which factors and which upstream delays produced that forecast — a prediction an operations team can act on and interrogate.

2. Data

The dataset is synthetic and fully reproducible (SEED = 42): 300 shipments across 10 ports (86 observed origin→destination routes), scheduled departures spanning 2026-01-01 to 2026-06-29, 28 raw columns.

Generation. For each shipment: origin/destination ports are drawn without replacement; the scheduled departure is drawn uniformly over the window; the scheduled port arrival is departure + 5 days ± 6 h jitter; scheduled customs clearance is port arrival + 1 day ± 4 h; the delivery promise is drawn from N(70 h, 34 h) (independent RNG stream, clipped to ≥ 6 h) added to the scheduled clearance. Processing durations (port, customs, inland) are U(12, 48) h; the four operational drivers are drawn independently: port_congestion, weather_severity, customs_complexity as integers 0–3, document_readiness as U(0, 1) (higher = paperwork more ready).

Causal propagation. Each stage's delay is a *local* severity effect plus an inherited fraction of the previous stage's delay plus noise, clipped at zero and then rescaled once by DELAY_SCALE = 6/24 = 0.25 (the chain is positively homogeneous, so the scale preserves every correlation while putting the canonical 6 h threshold at a usable ~40 % positive class):

```
departure_delay = max(0, (1 - document_readiness).5 + N(0, 1))
port_delay      = max(0, band(congestion) + 0.55·departure_delay + N(0, 1))
customs_delay   = max(0, band(complexity) + (1 - document_readiness).1.5
                      + 0.55·port_delay + N(0, 1))
inland_delay    = max(0, band(weather) + 0.55·customs_delay + N(0, 1))
```

where band(s) draws uniformly inside a severity→hours range (congestion and customs complexity {0:(0,1), 1:(1,3), 2:(3,6), 3:(6,10)}; weather {0:(0,0.5), 1:(0.5,2), 2:(2,5), 3:(5,9)}). The bands are deliberately modest: if the local term dwarfed the inherited term, stage-to-stage correlation would wash out and the dataset would lose the very propagation it exists to demonstrate. The realized stage-to-stage correlations are 0.314 (departure→port), 0.515 (port→customs), 0.582 (customs→inland).

Missing-data simulation. Each of the three intermediate actual timestamps (actual_port_arrival, actual_customs_clearance, actual_delivery) is nulled independently with probability 0.12 and paired with a 0/1 indicator column (realized rates 13.3 % / 13.7 % / 14.3 %). Because a stage delay is *defined* as actual – scheduled, port_delay_hours is nulled wherever the port-arrival timestamp is missing — it is a model *input*, and an operator at the cutoff genuinely cannot compute it from an unrecorded timestamp. The post-cutoff delays and both targets stay populated: labels are assembled post-hoc during reconciliation, so point-in-time correctness binds the inputs, not the labels.

3. ETA and delay definitions

Exact formulas used throughout the pipeline (and verifiable from the data):

```
total_delay_hours = departure_delay + port_delay + customs_delay + inland_delay
is_delayed       = total_delay_hours > 6          (canonical threshold)
actual_X         = scheduled_X + (cumulative delay through stage X)
predicted_eta    = scheduled_delivery + predicted_delay_hours
delay_probability = P(is_delayed = True | features) (classifier)
risk_level       = LOW (< 0.3) | MEDIUM (0.3–0.6) | HIGH (> 0.6)
```

Stage delays are rounded to 2 decimals *before* the label is derived, so is_delayed recomputed from the stored CSV always agrees with the stored label, and total == sum(stage delays) holds exactly in the file.

4. Feature engineering

build_features() allow-lists 13 cutoff-legal inputs (any post-cutoff column is excluded by construction, and a runtime assertion rejects both leaked and unvetted columns):

Feature	What it captures	Why it matters for propagation
route	origin → destination lane	per-lane baselines; unseen lanes fall back to a global mean
month, day_of_week	scheduled-departure calendar	booking-time seasonality / weekday effects
departure_delay_hours	how late the vessel left (observed)	root of the cascade, inherited by every later stage
port_delay_hours	how late the vessel docked (observed)	second observed deviation, propagating into customs/inland
port_congestion, weather_severity, customs_complexity	severity 0–3 per stage	local conditions added on top of the inherited delay
document_readiness	0–1 paperwork score (higher = better)	drives departure lateness and customs clearance time
port_arrival_missing	1 if the port timestamp is missing	marks rows whose port delay had to be imputed
cumulative_delay_so_far	departure + port delay at the cutoff	the quantity that propagates into the remaining stages
schedule_slack	planned window – planned work	whether inherited delay still fits the delivery promise
previous_stage_delay	most recent observed stage delay	becomes customs delay at a later cutoff, by design

The three **propagation-aware features** are the heart of the design. cumulative_delay_so_far is the causal state of the shipment at the cutoff — exactly the hours lost upstream that will compress customs and inland transit — and is the feature the propagation hypothesis says should dominate. previous_stage_delay carries the same value as port_delay_hours today but names the *concept* "most recently observed stage delay", so a later cutoff (after customs, say) can change its source without touching any consumer. schedule_slack is the only feature that looks forward: it subtracts the *planned* customs and inland work from the planned window between docking and the delivery promise, so it is computable at booking time and tells the model whether the delay already incurred still fits inside the promise. One consistency rule is enforced mechanically: where port_delay_hours was missing, the in-pipeline PropagationConsistentImputer fills it with the training-fold median and then *recomputes* cumulative_delay_so_far and previous_stage_delay from the filled value, so the canonical identities (cumulative = departure + port, previous = port) hold on every row the model ever sees.

5. Modeling approach

At N = 300 with 13 mostly low-cardinality features, tree ensembles are the right tool class: gradient boosting and random forests capture the nonlinear severity bands and the interactions the propagation chain implies (e.g. local severity × inherited delay) without any representation learning, they are data-efficient in the regime where a neural network would mostly memorize, and they expose `feature_importances_` — which the explainability layer builds on directly. Gradient boosting won the regression comparison on test MAE; the random forest won classification on test recall. Both classifiers use `class_weight="balanced"` because the positive class is the minority (40 %), and both heads run through the identical preprocessing pipeline (imputation → one-hot route + numeric passthrough) so differences between heads are attributable to the task, not the plumbing. A deep model was deliberately not used: with 300 rows it cannot learn anything an ensemble misses, it would forfeit the built-in interpretability the deliverable requires, and it adds tuning burden with no headroom to pay for it.

6. Delay propagation

The causal chain the generator encodes is exactly the mechanism the model is asked to recover:

```
departure █████0.55████ port █████0.55████ customs █████0.55████ inland █████ delivery
```

Worked example — first test-set row, shipment SHP-0172 (Busan → Hamburg, scheduled departure 2026-06-07 04:00): the vessel left 1.33 h late (`departure_delay_hours` = 1.33, driven by `document_readiness` = 0.084); $0.55 \times 1.33 \approx 0.73$ h of that inherited into the port stage, which finished 2.22 h late on top of a severe congestion draw; at the cutoff the shipment carries `cumulative_delay_so_far` = 3.55 h against `schedule_slack` = 78.31 h. The regressor forecasts 8.05 h of final delay (actual: 7.85 h) and the classifier puts the delayed-probability at 0.915 (HIGH) — and the narrative layer states the mechanism in one sentence: “A 1.33h departure delay contributed to a 2.22h port delay, bringing the shipment to 3.55h cumulative delay and leaving 78.31h of schedule slack for the remaining stages.”

7. Explainability approach and its stated limitation

Three explanation layers ship with the models, all in `src/explainability/explainer.py`:

- 1. **Global importances** — the final estimator's built-in `feature_importances_`, with all one-hot route__* indicators aggregated back into a single route column so tables match the 13-feature contract.
- 2. **Permutation importance** — `sklearn.inspection.permutation_importance` on the validation fold; for the regressor it ranks `port_delay_hours` (0.746 MAE loss) and `customs_complexity` (0.588) first, confirming the model uses the causal structure the generator built.
- 3. **Per-prediction contributors** — for one shipment, each numeric feature's raw contribution is `global_importance × (value - training-fold mean)`, the top-3 by magnitude are kept, and their absolute impacts are rescaled to sum toward the predicted delay; the propagation sentence narrates the two observed stage delays and remaining slack.

Stated limitation: `impact_hours` is a transparent, rescaled heuristic — it ranks and explains, but it is **not** a rigorous Shapley-value decomposition, and the code says so. With three near-duplicate delay features (`port_delay_hours`, `cumulative_delay_so_far`, `previous_stage_delay` share almost the same signal by construction), exact per-column attribution is ill-defined anyway; the heuristic favours readable, ranked explanations over false-precision numbers. A SHAP backend is the natural upgrade path.

8. Evaluation methodology

Models are compared on a **temporal split**: rows are sorted by `scheduled_departure` and cut chronologically into 210 train / 45 validation / 45 test, reproducing the deployment situation — fit on the past, score the future — and preventing the route-level information leakage a random split would quietly hand over. There is no cross-period leakage: every held-out row departs after every training row.

Why MAE for regression: the deliverable is “hours late”, and MAE is exactly that in units an operator understands (test MAE 0.648 h ≈ 39 minutes); it is also robust to the occasional large miss, unlike MSE, which would let a few extreme errors dominate model selection on a 45-row test set. RMSE and R² are reported alongside for completeness.

Why positive-class recall for classification: the cost of telling an operations team “all clear” about a shipment that arrives hours late far exceeds the cost of a false alarm, so the selection rule is *highest test positive-class Recall, tie-broken by F1*. Precision, F1 and ROC-AUC are reported alongside so the recall choice is visible and auditable.

9. Results

Fresh end-to-end run of `src/models/train.py` (temporal split, identical pipelines per head):

Regression — `total_delay_hours` (hours)

model	val MAE	val RMSE	val R2	test MAE	test RMSE	test R2
Route-mean baseline	1.795	2.316	-0.538	2.642	3.091	-0.514
Linear regression	0.732	0.960	0.736	0.733	0.920	0.866
Random forest	0.704	0.875	0.780	0.806	0.954	0.856
Gradient boosting	0.644	0.784	0.824	0.648	0.746	0.912

Selected: **gradient boosting** (lowest test MAE, 0.648 h ≈ 39 minutes) → `models/eta_regressor.joblib`.

Classification — `is_delayed` (positive class = delayed)

model	val Prec	val Recall	val F1	val ROC-AUC	test Prec	test Recall	test F1	test ROC-AUC
Majority-class baseline	0.000	0.000	0.000	0.500	0.000	0.000	0.000	0.500
Logistic regression	0.870	0.833	0.851	0.938	0.870	0.952	0.909	0.984
Random forest	0.800	0.833	0.816	0.915	0.875	1.000	0.933	0.980

Selected: **random forest** (test recall 1.000, F1 0.933) → `models/delay_classifier.joblib`. All held-out numbers are one draw on 45 test rows at a single seed and should be read with that uncertainty in mind.

10. Limitations

- **Synthetic data.** Every number in this report comes from a generated dataset, not real freight operations. The causal structure is real *within the generator*, but no claim is made about real-world effect sizes, route economics, or delay distributions.
- **Single fixed prediction cutoff.** The model predicts once, at port arrival. Real operations would re-predict at each milestone (departure, customs clearance, ...) as information arrives; the feature contract anticipates this (`previous_stage_delay`) but the pipeline implements only the one cutoff.
- **impact_hours is an approximation.** The per-prediction contributor breakdown is a rescaled importance heuristic, not a rigorous Shapley-value decomposition; with the three near-duplicate delay features an exact per-column attribution is ill-defined (§7).
- **Small N.** 300 rows, 45-row held-out sets, one seed. All reported metrics are one draw with wide uncertainty; nothing here supports claims about generalization to new lanes, seasons, or carriers.
- **No live data-source integration.** There is no feed of real vessel/port events; the API consumes hand-supplied payload fields.

11. Future improvements

- **SHAP-based explanations** to replace the rescaled heuristic with exact additive attribution per prediction.
- **Dynamic multi-stage re-prediction** as more milestones are observed — re-forecasting at customs clearance with `previous_stage_delay` sourced from the newly observed stage.
- **Validation against real AIS / port-call data** to test whether the propagation structure and feature importance ranking survive contact with real operations.
- **Persistent storage and a dashboard** so predictions, explanations and outcomes accumulate for monitoring, drift checks, and retraining.

12. Conclusion

This project built an end-to-end, reproducible freight-delay forecasting system: synthetic data with a genuine causal propagation chain, a leakage-enforced feature contract tied to a single well-defined prediction cutoff, gradient-boosting and random-forest models that predict both delay magnitude and delay risk, an explanation layer that ties every prediction back to the upstream delays that caused it, a FastAPI service, and a test suite that pins the whole contract. The central design decision was to invest in propagation-aware features and interpretable models rather than pursue a black-box accuracy-maximizing approach — at $N = 300$ the ensembles match or beat anything more complex would plausibly achieve, and the explanation deliverable (the entire point of the exercise) falls out of the model class instead of being bolted on. The result is a small system whose numbers can be questioned, checked, and re-derived — which is what makes it useful.

13. Example prediction walkthrough

A complete prediction, end to end, for a real held-out shipment — SHP-0172 (Busan → Hamburg), the first row of the test split, which the model never saw during training.

Input features (as observed at the port-arrival cutoff; actual outcome shown for scoring):

Feature	Value	Feature	Value
route	Busan → Hamburg	port_congestion	3
month	6	weather_severity	3
day_of_week	6 (Sunday)	customs_complexity	0
departure_delay_hours	1.33 h	document_readiness	0.084
port_delay_hours	2.22 h	port_arrival_missing	0
cumulative_delay_so_far	3.55 h	previous_stage_delay	2.22 h
schedule_slack	78.31 h	actual_total_delay_hours	7.85 h (delayed)

Calling `predict_shipment()` on that row returns:

```
{
  "predicted_delay_hours": 8.04843514741357,
  "predicted_eta": "2026-06-17T13:57:28.201716",
  "delay_probability": 0.915,
  "risk_level": "HIGH",
  "contributors": [
    {"factor": "schedule_slack", "impact_hours": 3.496598995537434},
    {"factor": "cumulative_delay_so_far", "impact_hours": 3.2785753190502667},
    {"factor": "customs_complexity", "impact_hours": -1.2732608328258699}
  ],
  "propagation": "A 1.33h departure delay contributed to a 2.22h port delay, bringing the shipment to 3.55h cumulative delay and leaving 4.33h remaining to be explained by the remaining features."
}
```

Reading it. The shipment left Busan 1.33 h late and docked 2.22 h late — about 0.73 h of the port delay inherited from departure at the generator's 0.55 coefficient, the rest from a severe congestion draw. By the cutoff the shipment carried 3.55 h of accumulated delay. The regressor turns that into a forecast of 8.05 h of final delay — against an actual of 7.85 h, an error of 0.2 h on a row it never trained on — and the ETA lands at 13:57 on 17 June versus the 05:54 delivery promise. The classifier flags the shipment HIGH with 0.915 probability of breaching the 6 h threshold, which it in fact did. The top contributors rank `schedule_slack` first (the row's slack is far from the training mean, so even a low-importance feature registers), then `cumulative_delay_so_far` — the propagation state itself — while `customs_complexity` *reduces* the estimate, correctly, because this shipment faces zero customs complexity. The propagation sentence states the mechanism in plain language, and the permutation importances (§7) confirm that this upstream-delay mechanism is what the model genuinely learned rather than a story bolted on after the fact.